
PROJECT REPORT

Project : ECE Pilot

Authors:

André-Marie

Alessandro

AMPÈRE

VOLTA

Teacher:

M. OHM

ABSTRACT

The development of autonomous vehicles is a major technological challenge, requiring high reliability and real-time processing capabilities. In the first semester, a remote-controlled rover was built using a "bare-metal" architecture. The problem addressed in this second semester is upgrading this system to a fully autonomous software architecture.

The main technical stakes involve migrating the control system to the Robot Operating System, establishing a reliable Micro-ROS bridge via Wi-Fi/Serial from an ESP32 microcontroller, and implementing real-time data fusion. Furthermore, the system must process this data to perform obstacle avoidance and Simultaneous Localization and Mapping.

This project is part of the ING4 Systems Engineering curriculum at ECE. It aims to provide practical experience in system integration, middleware application, and autonomous navigation algorithms over a full academic semester.

We certify that this submission is our own original work and meets the Faculty's Expectation and Originality.

City, DD/MM/YYYY

Table of contents

I. Goals	3
II. Glossary.....	3
A. Terms.....	3
B. Acronyms	3
III. The team.....	4
A. Presentation of the team.....	4
B. Team organization.....	4
C. GANTT diagram	4
IV. Context et problem.....	5
A. Context.....	5
B. Problem.....	5
C. Technical requirements	5
V. Conception.....	6
A. Functional architecture	6
B. Hardware architecture.....	6
C. Software architecture	7
VI. Development.....	9
A. Module 1	Error! Bookmark not defined.
B. Module 2	Error! Bookmark not defined.
C. Module 3	Error! Bookmark not defined.
VII. Tests & validation.....	10
A. Module 1	10
B. Module 2	10
C. Module 3	10
VIII. Assessment	10
A. Status of the project.....	10
B. Technical solution's relevance	10
C. Team management	10
IX. Bibliography	11
X. Appendices	12

I. Goals

The primary goal of this project is to develop a software stack enabling a Wave Rover to navigate a circuit entirely autonomously. To achieve this, the project focuses on three main objectives. First, we must establish a robust ROS 2 and Micro-ROS communication network between the hardware sensors and the main computer. Once the communication is stable, the next objective is to implement a reactive naive algorithm designed for autonomous obstacle avoidance using raw Lidar data. Finally, the system must deploy a SLAM algorithm to generate a reliable 2D map of the environment, which will serve as the foundation for computing an optimized trajectory in the final phase.

II. Glossary

A. Terms

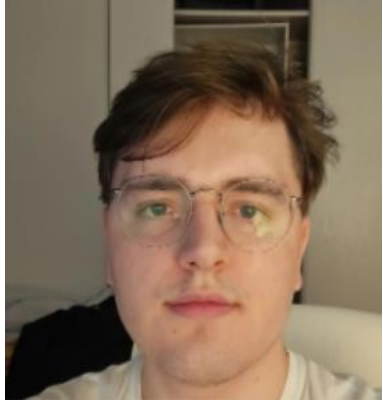
Term	Definition
Node	An executable that uses ROS to communicate with other nodes.
Topic	A named bus over which nodes exchange messages.
Middleware	Software that acts as a bridge between an operating system or database and applications.

B. Acronyms

Acronym	Meaning	Explanation
ROS	Robot Operating System	A software framework that standardizes communication between all of a robot's programs.
SLAM	Simultaneous Localization and Mapping	A technique allowing a robot to build a map of an unknown environment while tracking its position in real time.
IMU	Inertial Measurement Unit	A hardware sensor that measures the robot's acceleration and rotation.
LIO	Lidar Inertial Odometry	An algorithm that fuses Lidar distances and IMU data to accurately estimate the car's position on the track.
ESP-32	Espressif 32-bit microcontroller	Compute unit on the rover, supports WiFi and dual-core execution.

III. The team

A. Presentation of the team

Enguerrand	Evan	Clément
		
Polyvalent, serious and good for soldering	Organized, dynamic and good at reporting	Serious, good at coding and efficient

B. Team organization

An efficient organization was naturally established: We worked together for the first sprint with the installation of ROS2 and to better understand it, Clément and Enguerrand worked on the second sprint while Evan was taking care of the report. Then we worked together to fully develop the final part while we continue writing the report and the slides for the presentation. This flexible approach has allowed us to be involved in all aspects of the project.

C. GANTT diagram

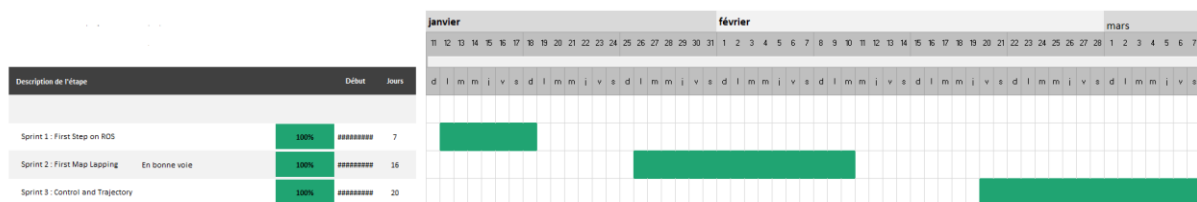
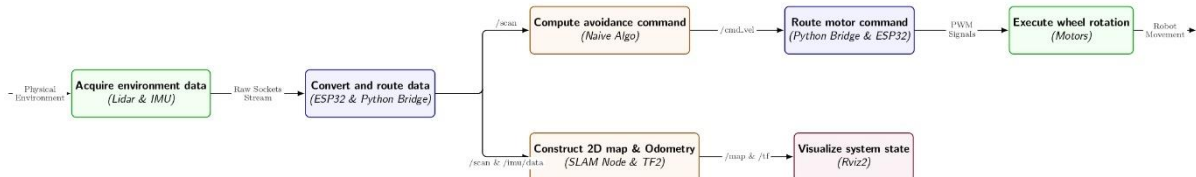


Figure : GANTT Diagram of the project



IV. Context et problem

A. Context

Autonomous driving requires modular and scalable architectures to process various sensor inputs simultaneously. While the previous semester focused heavily on hardware assembly and low-level control loops, this current phase shifts the focus toward high-level software abstraction. This transition is achieved by utilizing the industry-standard ROS 2 framework to manage complex interactions.

B. Problem

The core problem of this project is determining how to effectively transition from a monolithic embedded code structure to a distributed, node-based architecture. This new architecture must ensure the real-time processing of Lidar and IMU data to successfully achieve autonomous mapping and navigation, all while accommodating the limited computational resources of the onboard microcontroller.

C. Technical requirements

The project must adhere to several strict technical specifications. The entire system is required to operate using the ROS 2 Humble framework running on a dedicated Ubuntu environment. At the hardware level, the ESP32 microcontroller must function as a Micro-ROS client, taking responsibility for sending IMU data and executing received motor commands. For the vehicle's movement, the autonomous navigation must strictly process real-time sensor data rather than relying on any pre-recorded paths or hardcoded maneuvers. Finally, to ensure proper debugging and monitoring, all data logging and system visualization tasks must be fully compatible with visualization tools such as Rviz2 or Foxglove.

V. Conception

A. Functional architecture

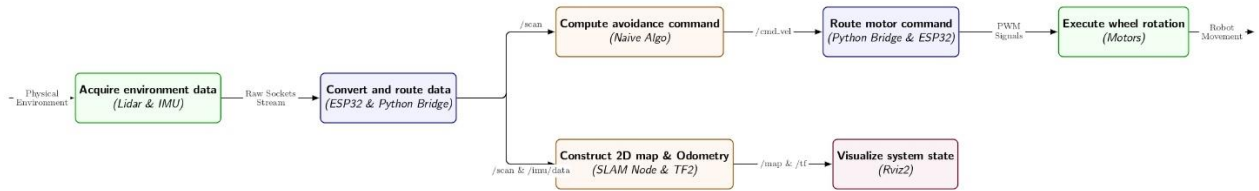


Figure : Functional Diagram

This functional diagram illustrates the continuous data flow within our autonomous system, mapping the logical sequence from environmental perception to physical action. To comply with functional design principles, the process is represented as a strict left-to-right sequence without backward physical loops.

The sequence begins with the acquisition of physical environment data via the Lidar and IMU sensors. This raw stream is then routed and converted by the ESP32 and our custom Python Bridge, which translates the socket data into standardized ROS 2 topics. The core intelligence of the system then splits into two parallel processing functions: the Naive Algorithm evaluates the immediate surroundings to compute obstacle avoidance commands, while the SLAM and TF2 nodes continuously construct a 2D map and track the rover's odometry.

Finally, the computed velocity commands are routed back through the Python Bridge to the hardware, where the motors execute the required wheel rotation, directly altering the robot's position in the physical environment. Simultaneously, the generated map and transforms are forwarded to Rviz2 to allow real-time visualization of the system's internal state.

B. Hardware architecture

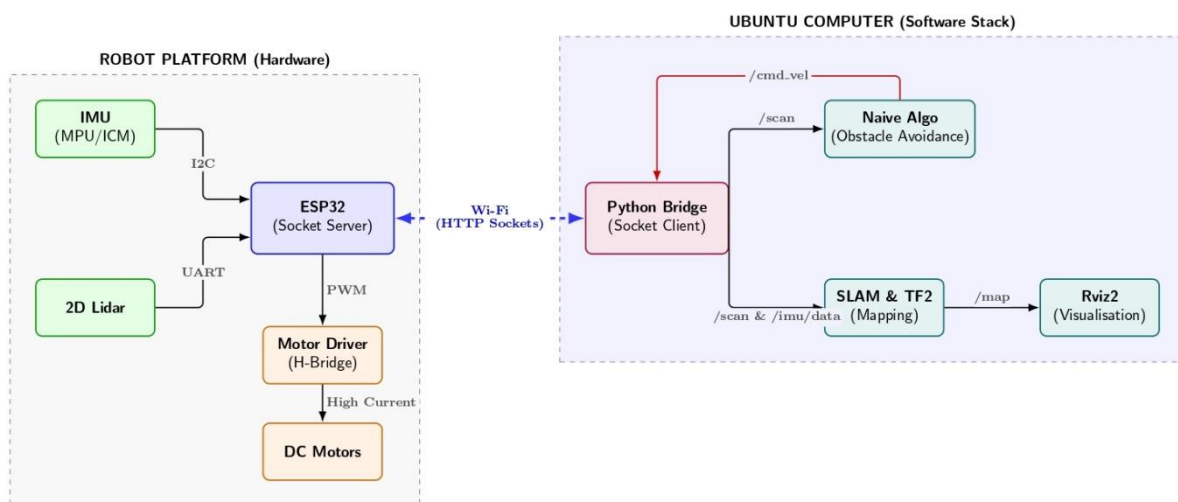


Figure : Hardware diagram

The hardware design is structured to minimize latency while maintaining a modular physical footprint. The core of the rover platform is the ESP32 microcontroller.

The Inertial Measurement Unit is connected directly to the ESP32 via the I2C protocol, providing high-frequency acceleration and rotational data. Similarly, the 2D Lidar is interfaced with the microcontroller via a UART connection to stream environmental distance measurements. For actuation, the ESP32 sends PWM signals to the motor drivers to control the speed and direction of the wheels.

All the collected data is packaged by the ESP32 and transmitted wirelessly via HTTP sockets over a local Wi-Fi network to the main processing computer, eliminating the need for a physical tether during autonomous navigation.

C. Software architecture

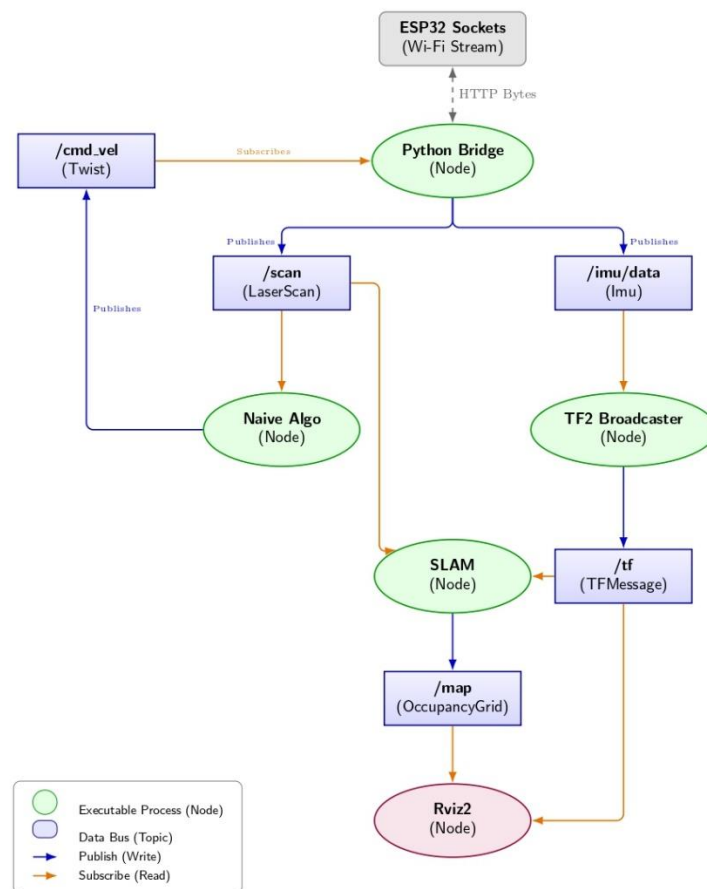


Figure : Software diagram

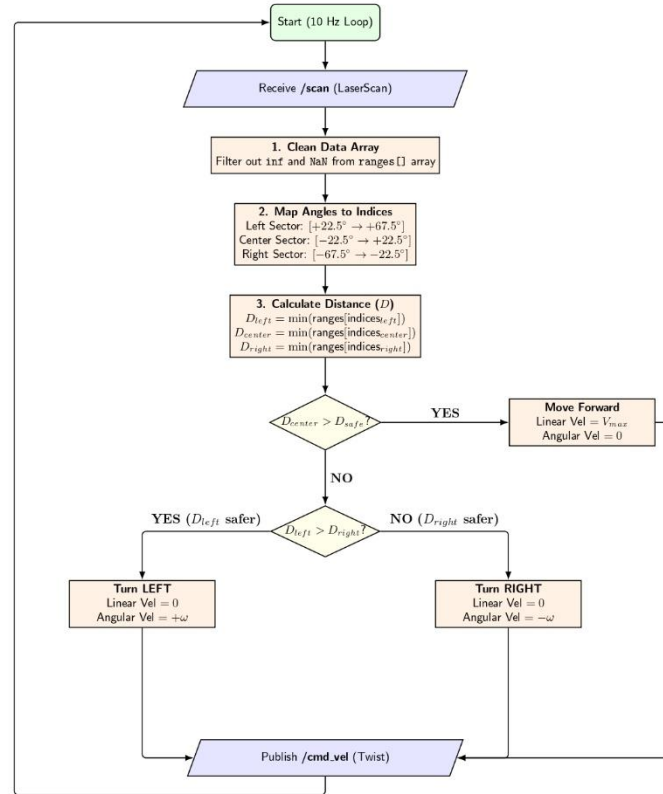


Figure : Naive Algorithm's diagram

The software architecture is entirely built upon the ROS 2 framework to ensure modularity. The entry point on the computer is the custom Python Bridge node, which continuously listens to the Wi-Fi sockets, unpacks the raw data, and publishes it onto the ROS 2 network as standard messages. Specifically, it publishes the distance data on the /scan topic and the orientation data on the /imu/data topic. To perform reactive autonomous navigation, a custom Naive Algorithm node, written in Python, subscribes to the /scan topic. This node divides the Lidar vision into three 45-degree sectors to detect obstacles. If the central path is blocked, the algorithm evaluates the left and right sectors to determine the safest route, subsequently publishing velocity commands on the /cmd_vel topic. These commands are then routed back through the Python bridge to the ESP32 to drive the motors.

VI. Development

A. ROS

The fundamental development of our autonomous architecture relies on the ROS, chosen for its distributed node-based ecosystem and real-time capabilities. A major technical decision was required regarding how to interface the hardware with this middleware.

Bypassing a standard, heavy Micro-ROS client directly on the ESP32 microcontroller was necessary to preserve its limited computational resources and prevent memory overflows. Instead, we developed a custom C++ HTTP socket server on the ESP32 to acquire and broadcast raw Lidar and IMU data over a local Wi-Fi network. On the Ubuntu computer, a dedicated Python bridge node was programmed to act as the socket client.

This custom bridge actively fetches the continuous byte streams, parses the payloads, and converts them into standardized ROS 2 messages. By publishing these structured messages to the `/scan` and `/imu/data` topics, this architectural choice significantly reduces latency and ensures a robust, asynchronous data flow, allowing all higher-level computational nodes to operate completely independently from the low-level hardware acquisition loop.

B. Naïve strategy and mapping

With the ROS network established, development shifted to environmental perception and reactive navigation. The naive strategy for obstacle avoidance processes the incoming Lidar arrays by dividing the forward-facing field of view into three 45-degree sectors: left, center, and right. To ensure maximum safety, a critical technical choice was made regarding the data treatment.

The algorithm does not calculate an average distance, which could mathematically obscure thin obstacles like table legs, but instead extracts the absolute minimum distance value within each sector array using the mathematical function. If the central distance falls below a hardcoded safety threshold, the robot halts its forward linear velocity and compares the left and right minimum distances to initiate a rotation toward the most open space.

Concurrently, the mapping block was developed to provide long-term spatial awareness. We implemented a TF2 broadcaster node to mathematically define the static spatial translation and rotation vectors between the robot's base, the IMU, and the Lidar. By feeding both the laser scans and the inertial odometry into a SLAM node, the system probabilistically computes the environment's boundaries, generating a real-time 2D occupancy grid on the `/map` topic that accurately reflects the physical test arena.

C. Ideal trajectory calculation

The final developmental stage upgrades the system from reactive, blind avoidance to proactive, intelligent navigation by calculating an ideal trajectory. Utilizing the 2D occupancy grid generated by the mapping module, the path planning architecture must compute a safe route from the robot's current odometry pose to a designated target coordinate.

To ensure the physical dimensions of the Wave Rover are respected during this calculation, a costmap algorithm is applied over the raw occupancy grid. This mathematically inflates the mapped walls by a radius equal to the robot's width, creating a vital safety buffer.

Once this navigable space is defined, a global path planning algorithm is utilized to calculate the shortest obstacle-free route, generating a sequence of spatial waypoints. Finally, a local trajectory controller translates this geometric path into physical kinematics. By analyzing the distance to the next waypoint and the current orientation, the local planner dynamically computes the optimal linear and angular velocities, publishing the final commands to the `/cmd_vel` topic to ensure the rover smoothly follows the ideal trajectory while compensating for real-time wheel slip.

VII. Tests & validation

This section is at least as important as the previous section.

The goal is to show the technical performances of your system and perform unit tests as well as integration tests in accordance with IV. part.

All figures or results (most of the times graphs) must be described as following:

- What has been done
- What we should obtain and what is the success criterion
- What we obtain

Conclusion: validation or not of the functionality.

A. Module 1

B. Module 2

C. Module 3

VIII. Assessment

A. Status of the project

Is the project fully finished? Are all the objectives and requirements fulfilled?

What remains to be done / improved?

B. Technical solution's relevance

What are the technical limitations of your solution?

How can we improve the performances of the system?

C. Team management

What did you individually learn? What are the new skills you can now put in your resume for your next internship research?

What can be improved in the team management? Propose an action plan.

IX. Bibliography

- [1] Dr Schneider, Maxime, "La Toolbox," [Online]. Available:
https://boostcamp.omneseducation.com/pluginfile.php/3538704/mod_resource/content/1/RAPPORT.pdf. [Accessed 14 juin 2024].
- [2] IEEE, "IEEE Editorial Style Manual," 29 février 2024. [Online]. Available:
<https://journals.ieeeauthorcenter.ieee.org/your-role-in-article-production/ieee-editorial-style-manual/>. [Accessed 14 juin 2024].

This section should list all the documents and websites used to develop the project. The syntax to use (IEEE bibliography norm) is described in the document « Comment rédiger un rapport » [1].

When they're cited in the document body, a numeric identifier must be added, in accordance with bibliography norm IEEE [2].

X. Appendices

Supplementary material and eventually some code (**no code in your report**).